

PROJECT 02

KYC Client Onboarding Service

Client Services — Know Your Customer

Graduate Software Engineering Programme 2026 · Poland — Q3 Cohort

PROGRAMME	Graduate Software Engineering Programme 2026 — Poland, Q3
BUSINESS AREA	Financial Services — Client Services
DELIVERY FORMAT	4 sprints — Foundations → API → Full Stack → Final Project
TECHNOLOGY STACK	SQL • Java • GitHub (Spring Boot, Docker optional)
PROJECT NUMBER	02 of 03
STARTER DATA	Data/ — sample DDL + seed SQL for domain exploration (not the solution schema)

Disclaimer. Additional requirements may be introduced during the programme (for example around tooling, demos, or stakeholder feedback). Treat this brief as the baseline — Sprint 4 is explicitly for folding those in and proving they are met.

1. Introduction

Know Your Customer (KYC) is a mandatory regulatory process requiring financial institutions to verify client identity and assess financial crime risk before providing services. This service manages the full onboarding lifecycle from initial application through to compliance sign-off.

Client Types	New and existing retail, corporate, and institutional clients; internal relationship managers; compliance and onboarding operations teams
Volumes	Hundreds of new client onboarding cases per day at a large bank; thousands of annual renewals
Frequency	KYC onboarding is event-driven (new client, trigger event, periodic review); periodic re-verification typically every 1–3 years depending on risk band
Regulatory	EU AMLD5/AMLD6; FCA KYC requirements; FinCEN Customer Due Diligence (CDD) Rule; FATF Recommendation 10

Inadequate KYC processes expose banks to financial crime risk and regulatory sanctions. The process must be thorough but efficient — slow onboarding frustrates clients and loses business. The system must maintain an auditable trail of every verification step and decision.

2. Sprint 1 — Foundations Challenge

Sprint 1 focuses on database design and administration first. By the end of this sprint the team should have a working, documented database with all core objects in source control. Java appears only as a thin proof-of-concept HTTP layer — not the focus of this sprint.

Starter data. Each project folder includes a `Data/ pack ( 00_ddl_schema.sql + numbered seed files + README)`. Use it to explore the domain and seed demos. It is **not** the answer to your E/R or §2.2 schema — table names and structure intentionally differ from this brief. Design your own model; you may map useful rows into it or invent your own seed inspired by the pack.

2.1 E/R Model — Entities

The following entities form the core data model. Trainees should produce a full E/R diagram as part of the deliverable.

E/R Entities	Client · ClientAddress · DocumentType · Document · RiskClassification · OnboardingCase · OnboardingStep · ComplianceOfficer
---------------------	---

2.2 Suggested Schema

Key tables, views, and stored procedures to implement. Trainees are encouraged to extend this where it makes sense for their domain.

Object	Type	Key Fields / Notes
Client	Table	client_id, full_name, client_type (INDIVIDUAL/CORPORATE), nationality, date_of_birth, tax_id, status (PENDING/ACTIVE/SUSPENDED/REJECTED)
ClientAddress	Table	address_id, client_id, address_type (REGISTERED/MAILING), street, city, country, postcode, effective_date
DocumentType	Table	doc_type_id, doc_type_name (PASSPORT/DRIVING_LICENCE/UTILITY_BILL/COMPANY_REG), required_for_individual, required_for_corporate
Document	Table	doc_id, case_id, doc_type_id, submission_date, verified_flag, verified_by, verified_at, expiry_date, rejection_reason
OnboardingCase	Table	case_id, client_id, opened_date, status, assigned_officer_id, due_date, completed_date
RiskClassification	Table	classification_id, case_id, risk_level (LOW/MEDIUM/HIGH), classification_date, assessed_by, rationale, next_review_date
pending_cases_vw	View	Cases with status OPEN or AWAITING_DOCUMENTS ordered by due_date
expiring_documents_vw	View	Documents expiring within 60 days across all active clients
open_case	Stored Proc	Create new client and onboarding case; auto-assign required document checklist
submit_document	Stored Proc	Record document submission; update case step status
classify_risk	Stored Proc	Apply risk classification to a case; update client risk band
complete_onboarding	Stored Proc	Mark case complete; activate client; record completion timestamp

2.3 Shell Scripts

As part of the Foundations deliverable, teams should produce the following shell scripts for basic DBA operations.

db_dump.sh	Full schema and data dump
db_reload.sh	Drop and reload from dump file
rebuild_indexes.sh	Rebuild all indexes

expiring_docs_report.sh	Run expiring_documents_vw and export to CSV
-------------------------	---

2.4 Thin Java API (POC)

Add a **very simple** Java HTTP layer that proves the database works end-to-end. This is a proof of concept only — 2–3 read endpoints, no full business logic, and no full CRUD. Any approach is fine (raw sockets, a micro-framework, or Spring Boot if preferred).

Method	Endpoint	Purpose
GET	/api/onboarding/cases/{id}	Return case details including document checklist (from the database)
GET	/api/clients/{id}	Return client by internal ID

You should be able to start the process and hit these endpoints with `curl`. That is enough for Sprint 1.

2.5 Sprint Deliverables

- E/R diagram in repo
- DDL script for all DB objects
- Shell scripts in `/scripts/db/`
- Test data: 10 clients (mix of individual/corporate) at various stages of onboarding
- Thin POC API (read endpoints above) that can be started and exercised with `curl`
- README.md with schema and setup instructions

3. Sprint 2 — API Challenge

Sprint 2 delivers a fully functional HTTP API covering all domain operations. Business logic belongs in Java — not only in the database. Choose any Java web approach that fits; Spring Boot is a suggestion, not a requirement.

3.1 Functional Specification

The API should implement the following capabilities:

Open Onboarding Case	Create a client record and onboarding case; auto-populate required document checklist based on client type
Submit Document	Record a document submission against a case; validate document type is on the checklist
Verify Document	Mark a document as verified; check all required documents are now verified
Classify Risk	Apply risk classification rules based on client type, nationality, and document profile; record rationale
Advance Case Status	Move the case through its lifecycle; validate all prerequisites are met for each transition
Flag Expiring Documents	Return all clients with documents expiring within a specified number of days

3.2 REST API

Suggested endpoints. Teams should implement all core endpoints and are encouraged to add domain-specific endpoints where appropriate.

Method	Endpoint	Purpose
POST	/api/clients	Create a new client record

POST	/api/onboarding/cases	Open an onboarding case for a client
GET	/api/onboarding/cases/{id}	Get case details including document checklist
POST	/api/onboarding/cases/{id}/documents	Submit a document for a case
PATCH	/api/onboarding/cases/{id}/documents/{docId}/verify	Mark a document as verified
PATCH	/api/onboarding/cases/{id}/status	Update case status
GET	/api/onboarding/cases?status=PENDING_REVIEW	List cases awaiting review
GET	/api/clients/expiring-documents?days=30	List clients with documents expiring within N days

3.3 Test Coverage

The following test types should be implemented using JUnit 5 and Mockito. Aim for meaningful coverage of business logic, not just line coverage.

Test Type	What is Tested
Unit – Risk Classification	Test LOW/MEDIUM/HIGH assignment rules for different client types, nationalities, and document combinations
Unit – Document Checklist	Test that correct documents are required for INDIVIDUAL vs CORPORATE clients
Unit – Expiry Detection	Test documents expiring in exactly N days, N-1 days, and N+1 days against threshold
Mock – Case State Machine	Mock repository; verify OPEN → AWAITING_DOCUMENTS → IN_REVIEW → APPROVED/REJECTED transitions
Mock – Document Submission	Mock repo; verify checklist item updated when correct document type submitted
Parameterised	Submit wrong document type; attempt to approve case with unverified documents; reopen rejected case

3.4 Logging

Use clear, structured logging with consistent levels (INFO / WARN / ERROR). A library such as SLF4J with Logback is a **suggestion**, not a requirement. Log entries should include timestamp, level, source, and a structured message. Examples:

Level	Example Log Message
INFO	Onboarding case opened: caseId=KYC-0441 clientId=C-9921 type=CORPORATE assignedTo=officer-003 due=2026-08-21
INFO	Document submitted: caseId=KYC-0441 docType=COMPANY_REGISTRATION submittedBy=jsmith
INFO	Document verified: caseId=KYC-0441 docType=COMPANY_REGISTRATION verifiedBy=officer-003
INFO	Risk classification: caseId=KYC-0441 level=MEDIUM rationale=NonEEA corporate, missing UB0 documentation
WARN	Document expiring in 14 days: clientId=C-9921 docType=PASSPORT expiry=2026-08-21
WARN	Case overdue: caseId=KYC-0398 dueDate=2026-08-05 currentDate=2026-08-07 daysOverdue=2
ERROR	Invalid state transition: caseId=KYC-0441 from=OPEN to=APPROVED – documents not fully verified

3.5 Additional Requirements

- Machine-readable API documentation for all endpoints (e.g. OpenAPI / Swagger, or equivalent)
- A health (or readiness) endpoint that reports whether the service can accept traffic
- Document expiry check as a scheduled job (daily at 07:00)

Optional suggestions (not required). Teams may use Maven or Gradle, Spring Boot (or another Java web framework), Docker / docker-compose, and SLF4J with Logback if they want — these are welcome but not graded as mandatory deliverables.

3.6 Sprint Deliverables

- Full HTTP API for all core operations (framework of your choice)
- Automated test suite covering the business logic (e.g. JUnit 5 + Mockito)
- Consistent structured logging across the service
- API documentation with example requests and responses
- README covering how to build, run, and call the API (e.g. with curl)

4. Sprint 3 — Full Stack Challenge

Sprint 3 delivers the **finished working application** — feature-complete end-to-end. Database, Java services, HTTP API, and a simple frontend should hang together as one runnable product. By the end of this sprint the app itself is done; Sprint 4 is for polish, compliance with the brief, and presentation.

Duration	Focused build days within the sprint calendar
Team Size	2–3 developers
Repository	Single shared GitHub repo per team

4.1 Sprint Deliverables

- Working end-to-end application integrating database, Java service, HTTP API, and frontend — all core flows runnable
- Minimum frontend screens listed in §4.2, wired to the live API (not mocked-only)
- Automated CI pipeline (build + test) that runs on every push
- Complete API documentation for all endpoints
- Developer onboarding guide — a new team member should be able to clone, build, and run in under 10 minutes
- Structured logging throughout, with log levels used consistently
- Evidence of collaborative working: pull requests, code reviews, branch strategy
- README covering architecture, setup, design decisions, and known limitations

4.2 Frontend (minimum UI)

Build a small browser UI that an onboarding / compliance officer could use day-to-day. Keep scope tight — plain HTML/CSS/JS or a lightweight framework are both fine.

Case queue	List onboarding cases with status and due date; filter by at least one status (e.g. open / awaiting documents)
Case workspace	Open a case: show client summary, risk band (if present), and document checklist / submission status from the API

Simple action	One write action from the UI (e.g. assign officer, mark a document verified, or advance case status) that persists via the API
----------------------	--

Presentation & marks. Meeting the screens above is enough to pass the frontend bar. A well presented, easy-to-read, easy-to-navigate UI earns **additional points** — clarity and usability count beyond bare functionality.

Optional suggestions (not required). Teams may use Maven or Gradle, Spring Boot (or another Java web framework), Docker / docker-compose, and SLF4J with Logback if they want — use them where they help; they remain optional through to the final presentation.

5. Sprint 4 — Final Project

Sprint 4 assumes the application from Sprint 3 is already complete. The focus shifts to **making sure every requirement is met** (from earlier sprints and any new ones introduced during the programme), polishing the product, and preparing the presentation. At the end of Sprint 4 the final project is done.

Duration	Final days of the project calendar
Team Size	2–3 developers
Presentation	Final day — 15 minutes to managers and the graduate programme team
Repository	Same shared GitHub repo — presentation-ready

5.1 Sprint Deliverables

- Requirements walkthrough: evidence that Sprint 1–3 deliverables (and any later additions) are all satisfied
- Polished demo path — stable scripts or steps to show the main flows without friction
- Innovation component ready to explain and demonstrate
- Final README and onboarding guide checked for accuracy
- Presentation materials and speaking plan for the structure below

5.2 Innovation Component

Each team should include at least one innovative use of technology relevant to their domain. This could include — but is not limited to — integration of a Gen AI component (e.g. anomaly explanation, natural language query, automated summary), a real-time data feed simulation, an event-driven architecture using a message queue, or a performance benchmark with analysis.

Teams should be prepared to explain their innovation choice and demonstrate it during the presentation.

Optional prompts for this project (pick at most one, or invent your own — suggestions only, not mandatory):

- Gen AI case brief — auto-summarise a case (client, docs, risk) for the compliance officer before sign-off
- Document expiry / renewal dashboard with a simulated reminder when a passport or utility bill is near expiry
- Risk-band explainer — show which factors (country, PEP, product, etc.) drove the classification in plain language

5.3 Presentation Structure

Total length: **15 minutes**. Suggested sections (timebox within that total as you see fit):

Architecture Overview	System diagram and key design decisions
Live Demo	Run the application and show the main flows
Code Walkthrough	One area of code the team is proud of
Innovation	Explain and demo the innovation component
Retrospective	What worked and what you would do differently

Good luck — build something you're proud of.